A. Introduction to ARM Cortex Mx Processor

The Family of ARM Cortex Processors is presented in the Figure 1. The Lecture Note emphasizes on architecture, Memory system, exceptions, and Interrupts for ARM Cortex Mx series of processors only. The design start of Cortex M0 processor is shown in the Figure 2.

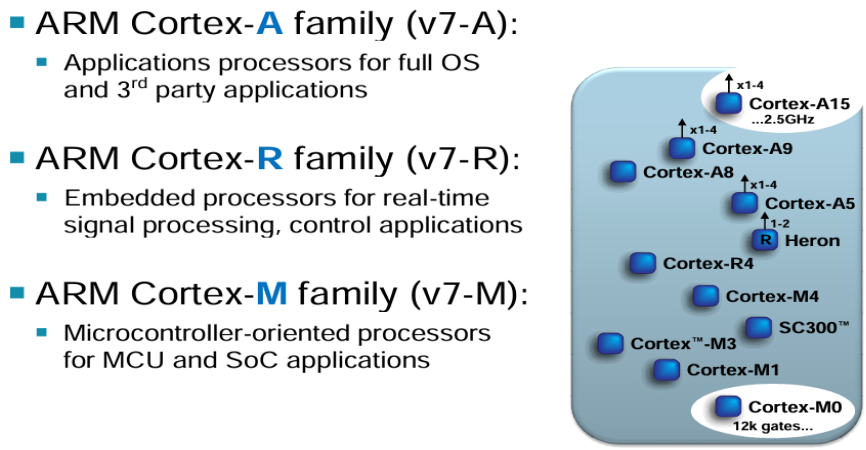


Fig 1. Family of ARM Cortex A/R/M processors.

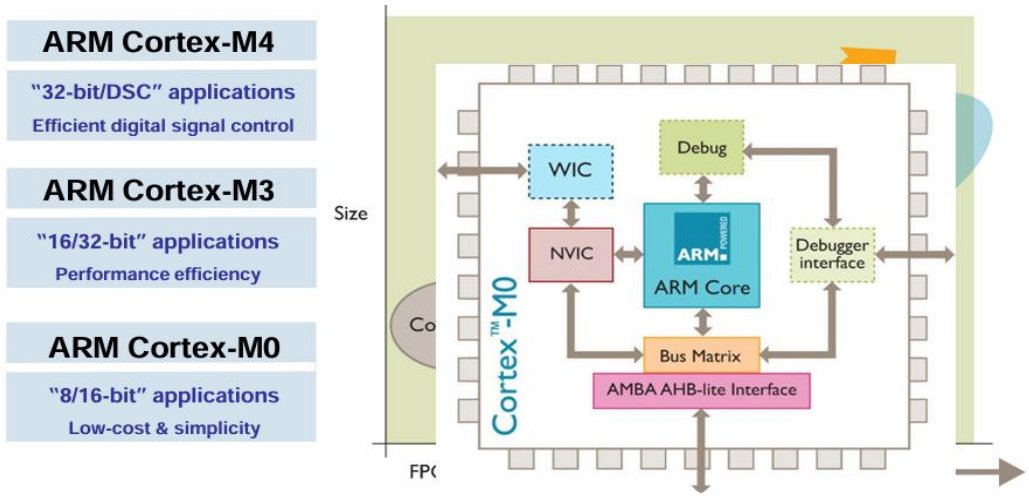


Fig 2. Design Start of ARM Cortex M0 Processor.

A.1 Characteristics of ARM Cortex Mx Processors

It is a Harvard architecture because it has separate data and instruction buses. The Cortex-M instruction set combines the high performance typical of a 32-bit processor with high code density typical of 8-bit and 16-bit microcontrollers. Instructions are fetched from flash ROM using the Instruction Code bus. Data are exchanged with memory and I/O via the system bus interface. On the Cortex-M4 there is a second, I/O bus for high-speed devices like USB. There are many sophisticated debugging features utilizing the Data Code bus. The nested vectored interrupt controller (NVIC) manages interrupts, which are hardware-triggered software functions. Some internal peripherals, like the NVIC communicate directly with the processor via the private peripheral bus (PPB). The tight integration of the processor and interrupt controller provides fast execution of interrupt service routines (ISRs), dramatically reducing the interrupt latency.

Key Building blocks of the ARM Cortex Processor is presented below.

- Instruction Set Architecture
- Register Set
- Processor Operating Modes
- Interrupts and Processor Reset Sequence
- Pipelined Architecture and Data Path
- Memory Address Map

The Block diagram consisting of the above key components is shown in the Figure 3. The architectural difference between Cortex M3 and M4 cores are highlighted in the Figure 4.

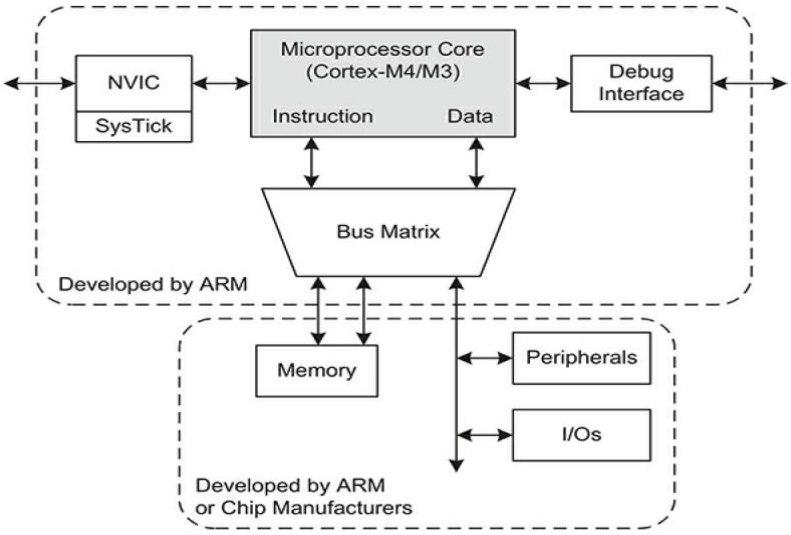


Fig 3. Architectural Block Diagram for ARM Cortex M processor

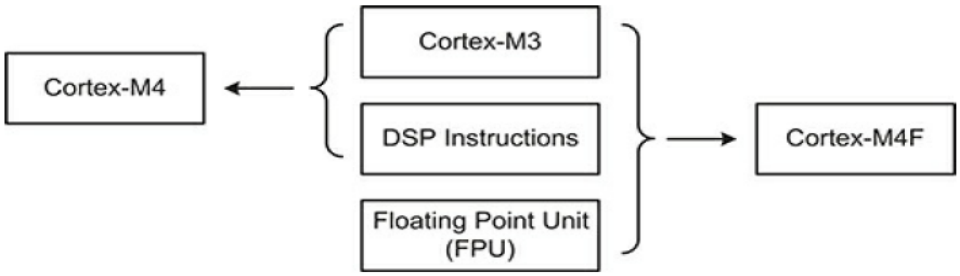


Fig 4. Architectural Difference between ARM Cortex M3 and M4 processors

A.2 ARM Instruction Set Architecture

ARM processors have supported two different instruction sets: the ARM instructions that are 32 bits and Thumb instructions that are 16 bits. The size of an instruction (i.e., assembly instruction) signifies the number of bits required to store the machine code or opcode of that instruction. From functionality provisioning viewpoint, the Thumb instruction set is a subset of the ARM instructions. Using Thumb instructions can provide higher code density. It is a preferable choice for products with tight memory requirements. On the other hand, using ARM instructions can improve the processor execution performance.

Thumb2 is a superset of the Thumb instruction set. Thumb2 introduces 32-bit instructions that are intermixed with the 16-bit instructions. The Thumb2 instruction set covers all the functionality of the Thumb instruction set. Thumb2 has the execution performance close to that of the ARM instruction set and has the code density performance close to the original Thumb Instruction Set Architecture (ISA). Cortex-M3 is primarily focused on small memory system devices such as microcontrollers and aiming to reduce the size of the processor, supports only the Thumb2 instruction set.

A.3 Register Set

The data processing related registers are responsible for temporary storage of data elements before and after the execution of instructions. In addition, control registers are responsible for configuring the desired mode of operation of the processor, while the status registers provide information regarding the current state as well as mode of operation. The Cortex-M3 processor has 32-bit registers, which are shown in Figure 5.

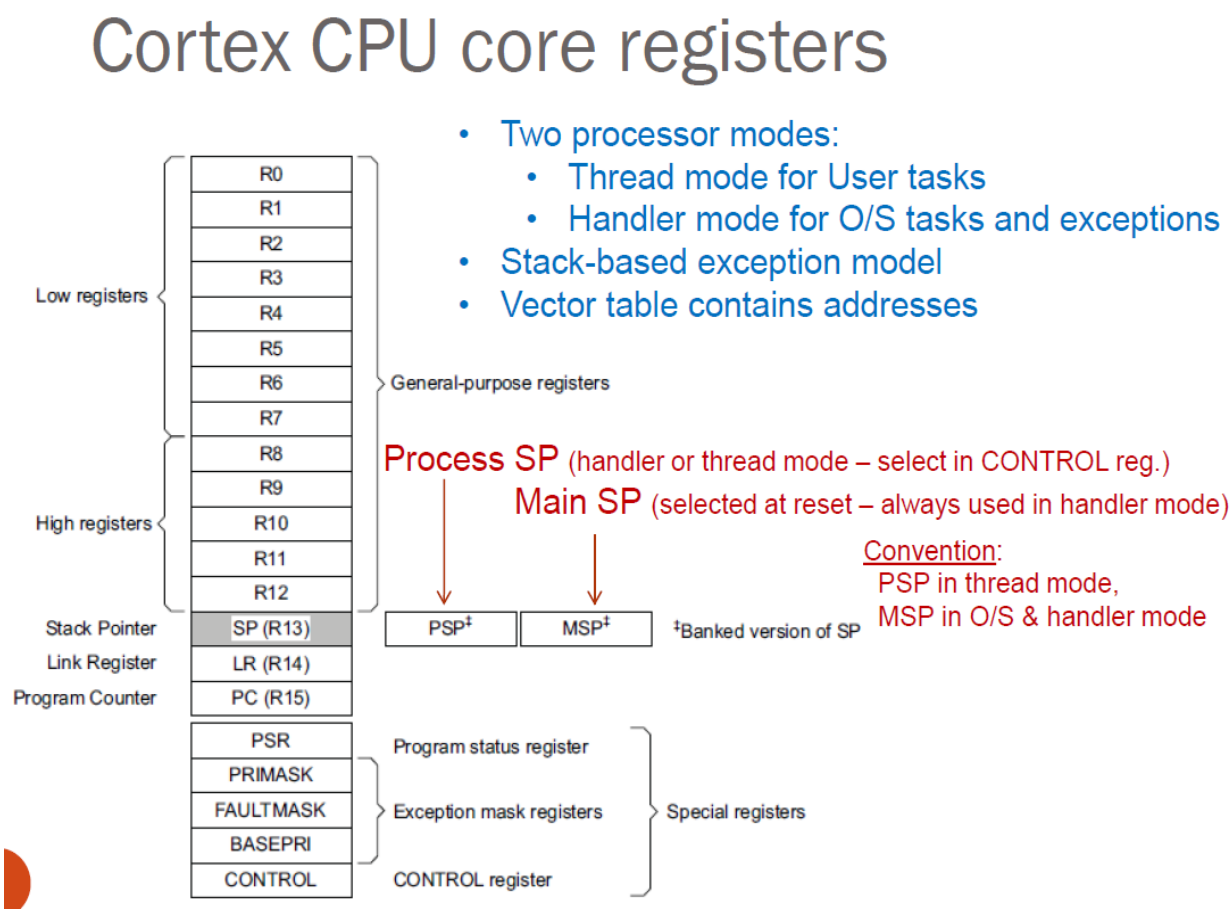


Fig 5. Register Set of the ARM Cortex M3 Processor.

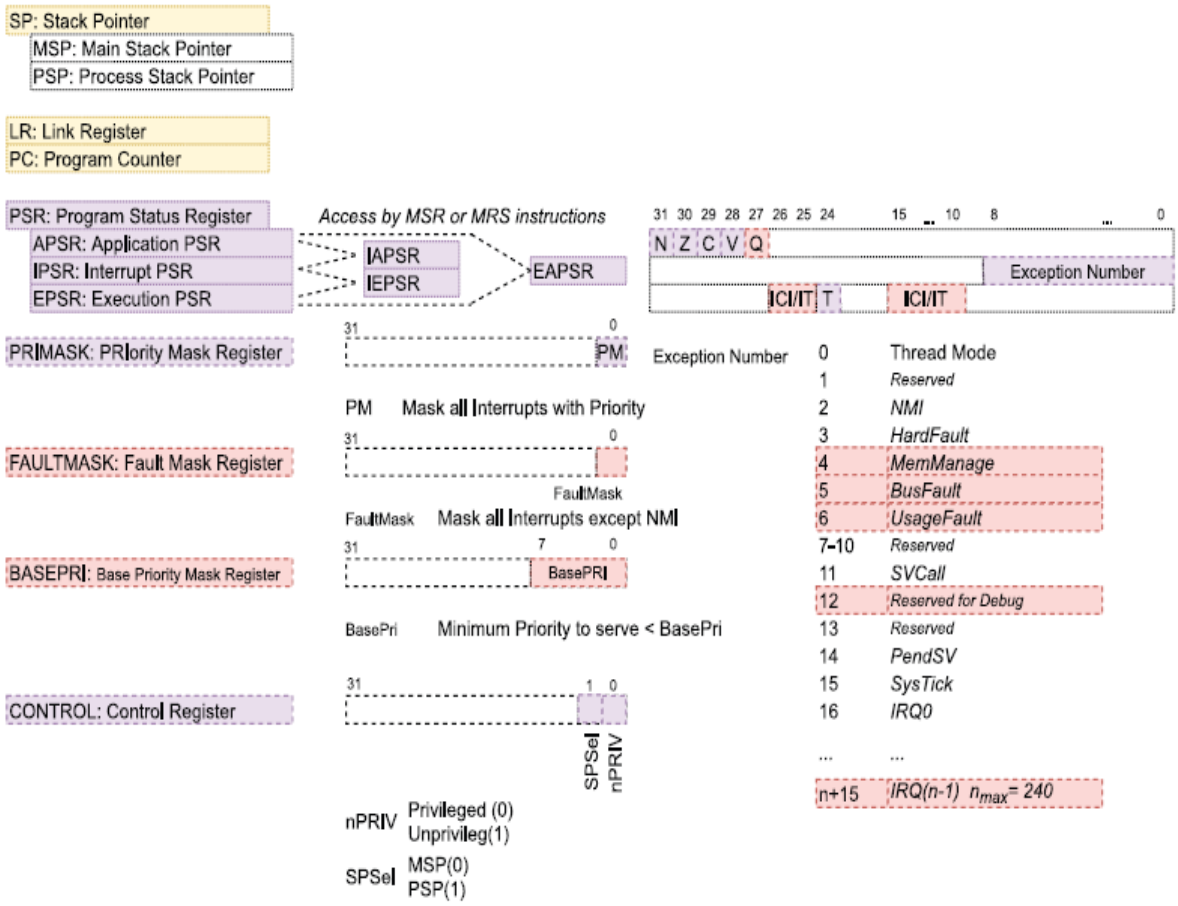


Fig 5 Contd. Summary of all special and status registers.

- General-purpose registers, R0-R12, where Registers R0-R7 are called low registers and are accessible by all instructions that specify a general-purpose register, and Registers R8-R12 are called high registers and are accessible by all 32-bit instructions that specify a general-purpose register. Registers R8-R12 are not accessible by any Thumb (16-bit) instructions.
- Stack Pointer (SP): Register R13 is used as the Stack Pointer (SP). Because the SP ignores any writes to bits [1:0], this makes it aligned to a word (4 byte) boundary automatically. In addition, it is important to remember that stack pointer is a banked register with two copies, namely Main Stack Pointer (MSP) and Process Stack Pointer (PSP). Only one copy of the stack pointer (R13) is visible and active at a given time. This means that stack pointer logically has one copy at any arbitrary time instant, while physically it has always two copies. Handler mode always uses MSP, but you can configure Thread mode to use either MSP or PSP.
- Link Register (LR): Register R14 is the subroutine Link Register (LR). The LR receives the return address from the program counter register, discussed next. The link register contains the return address to be used by the processor, when returning from a function or service routine. When the link register is not used for holding a return address, it can be treated as a general-purpose register.
- Program Counter (PC): Register R15 is called the program counter register. PC contains the current program or instruction address that is to be executed. This register can be modified by the program itself to control the flow of the program. Bit 0 of this register is always 0, which ensures that the instructions are always aligned to either word or halfword boundaries in the code memory.

Program Status Registers

This set of Special registers consists of the following three status registers:

1. Application Program Status Register (APSR)

2. Interrupt Program Status Register (IPSR)

3. Execution Program Status Register (EPSR)

These registers have special functions and can be accessed only by special instructions. They cannot be used for normal data processing. Figure 6 shows different bit fields used by program status register and the definitions of these bit fields are explained in Table 1.

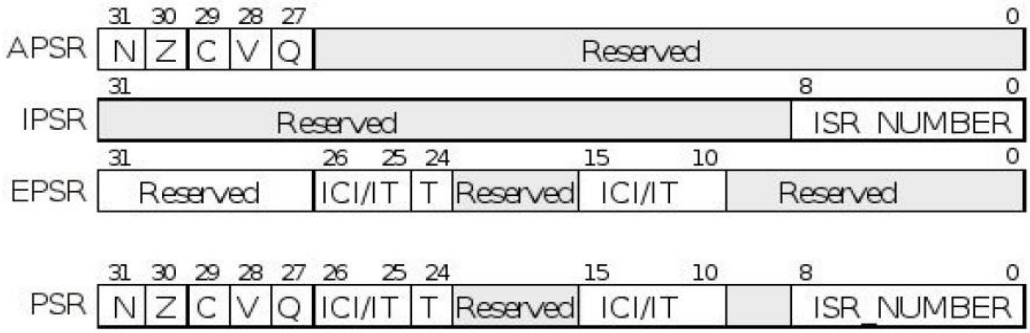


Figure 6. Program status registers and their bit field allocations

Table 1. Bit field definitions for different program status registers

Bit field	Description
N	Negative flag.
Z	Zero flag.
C	Carry flag.
V	Overflow flag.
Q	Saturation flag.
ICI/IT	Interrupt continuable instruction (ICI) bits. IF-THEN instruction block status bit.
T	Thumb state (always 1 in case of Cortex-M processor using Thumb2 ISA).
Exception number	Shows which exception number is being handled by the processor.

The ARM Architecture Procedure Call Standard, AAPCS, part of the ARM Application Binary Interface (ABI), uses registers R0, R1, R2, and R3 to pass input parameters into a C function. Also, according to AAPCS we place the return parameter in Register R0.

There are three status registers named Application Program Status Register (APSR), the Interrupt Program Status Register (IPSR), and the Execution Program Status Register (EPSR) as shown in Figure 6. These registers can be accessed individually or in combination as the Program Status Register (PSR). The N, Z, V, C, and Q bits give information about the result of a previous ALU operation. In general, the N bit is set after an arithmetical or logical operation signifying whether the result is negative. Similarly, the Z bit is set if the result is zero. The C bit means carry and is set on an unsigned overflow, and the V bit signifies signed overflow. The Q bit is the sticky saturation flag, indicating that "saturation" has occurred, and is set by the SSAT and USAT instructions.

The T bit will always be 1, indicating the ARM ® Cortex-M is executing Thumb instructions. The ICI/IT bits are used by interrupts and by the IF-THEN instructions. The ISR_NUMBER indicates which interrupt if any the processor is handling. In Figure 5, Bit 0 of the special register PRIMASK is the interrupt mask bit. If this bit is 1 most interrupts

and exceptions are not allowed. If the bit is 0, then interrupts are allowed. The Bit 0 of the special register FAULTMASK is the fault mask bit. If this bit is 1 all interrupts and faults are not allowed. If the bit is 0, then interrupts and faults are allowed. The non-maskable interrupt (NMI) is not affected by these mask bits. The BASEPRI register defines the priority of the executing software. It prevents interrupts with lower or equal priority but allows higher priority interrupts. For example, if BASEPRI equals 3, then requests with level 0, 1, and 2 can interrupt, while requests at levels 3 and higher will be postponed.

A.4 Processor operating modes

The processor supports two modes of operation, Thread mode and Handler mode.

Thread Mode: The processor enters Thread mode on reset or because of an exception return. Code execution in thread mode can have either privileged or unprivileged (user) access levels.

Handler Mode: The processor enters Handler mode because of an exception. All code is privileged in handler mode.

In thread mode with unprivileged (user) access level, permission to modify the contents in system control space (SCS), which is a part of the memory region for configuration registers and debugging components, is blocked. Furthermore, instructions that access special registers cannot be used. If a program running at the user access level tries to access SCS or special registers, a fault exception will occur. When an exception takes place or an interrupt occurs, the processor will always switch to a privileged access level and returns to the previous state when exiting the exception handler. A program with user access level cannot change to the privileged level directly by writing to the control register. As mentioned previously, it must go through an exception handler which takes it to privileged access level. However, the program running at the privileged access level can always switch the processor execution level to the unprivileged access level by writing to the control register. The possible state transitions from one operating mode to the other as well as the corresponding privileged/unprivileged access levels at each state are shown in Figure 7.

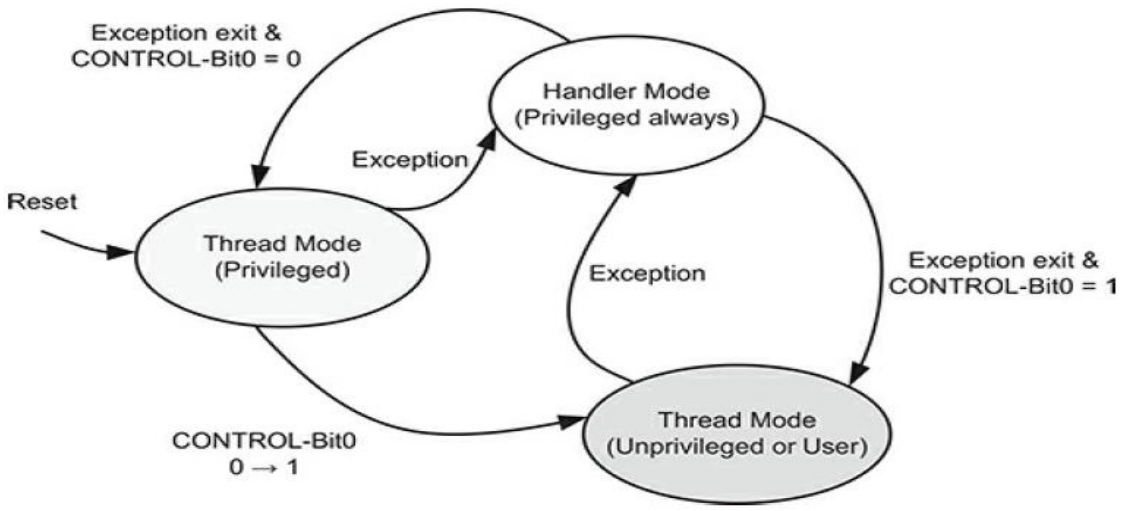


Fig 7. Transitions for Processor operating modes

These possible combinations based on the CONTROL-Bit0 and CONTROL-Bit1 values are given in Table 2.

A.5 Interrupts and Processor Reset Sequence

An interrupt causes a deviation from the normal sequence of program execution. In case of an event or exception, it is natural to think that a response is required from the processor. The processor response generally involves a sequence of operations to be performed and is implemented as a dedicated response function and is called

service routine. The sequence of operations involved from the occurrence of an event to the resumption of normal program execution again has the following main steps.

- When an event occurs, an interrupt request is generated to the processor.
- The processor suspends current task execution in response to the interrupt and starts executing an interrupt service routine to generate the response to the event.
- Once the response to the event is completed, the processor resumes execution of the suspended task.

When a microcontroller is powered or if it reset by pressing the reset button, an interrupt called reset interrupt is generated. Every processor has an associated sequence of operations, which it performs to reset event. The first operation performed by the Cortex-M processor after a reset or power-on involves reading of two words from instruction memory.

Address 0x00000000: This address in the code memory (most likely a Flash memory) contains the starting value that is loaded to the main stack pointer, MSP (R13).

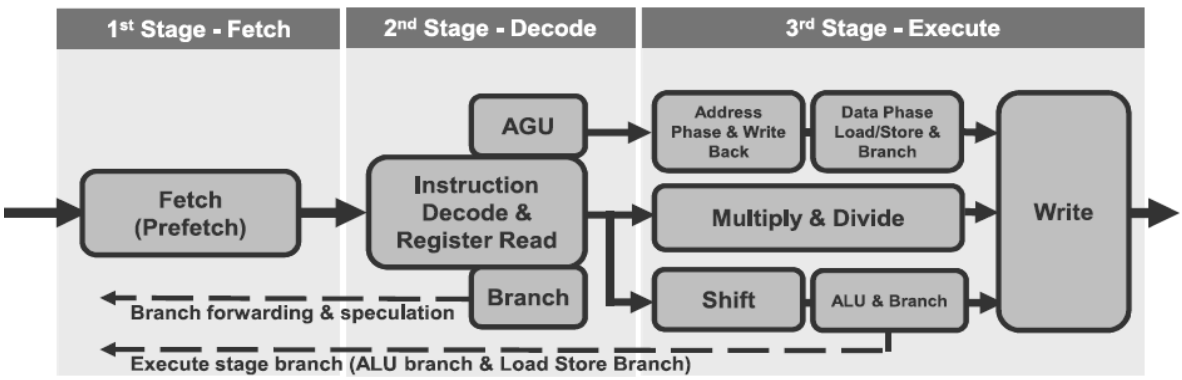
Address 0x00000004: Reset vector is contained at this address and the program counter is loaded with this value to jump to the reset interrupt service routine. The main function is called from within reset interrupt service routine.

Table 2. Possible Combinations and Usage based on Ctrl -bit 0 and Ctrl-bit 1

Control Bit 0	Control Bit 1	Descriptions
0	0	Entire program runs at privileged level and only the Main Stack Pointer (MSP) is used by both the main program as well as interrupt handlers
0	1	The user application uses Process Stack Pointer (PSP), while the MSP is used by the operating system kernel and exception or interrupt handlers.
1	0	It is highly unlikely that this scenario will be used by the user applications, where the user application and the operating system share the stack memory.
1	1	The user application uses Process Stack Pointer (PSP), while the MSP is used by the operating system kernel and exception or interrupt handlers.

A.6 Pipelined Architecture and Data Path

- Cortex-M3 has 3-stage fetch-decode-execute pipeline
  - Similar to ARM7
  - Cortex-M3 does more in each stage to increase overall performance



A.7 Memory Address Map

Based on the preferred memory address space usage, we can categorize the memory space in the following regions:

- Code region for storing program or code. In general, a Flash or EEPROM memory is in this address space.
- Data region for application data. Mostly SRAM is placed in this region.
- Peripheral region hosts different input and output peripherals.
- External memory and peripheral region, through a bus interface, can have access to external memory or peripherals.
- Private Peripheral Bus (PPB) (both internal and external) region is meant for debug interfaces. This address space also hosts the registers for the Nested Vectored Interrupt Controller, processor's configuration registers, as well as registers for debug components. Note that the Private Peripheral Bus (PPB) is not used for normal peripherals. This will be further explained in this chapter later.
- Vendor specific region contains information about device ID, versions, release, etc.

The Predefined memory map for ARM Cortex Mx is shown in Figure 8.

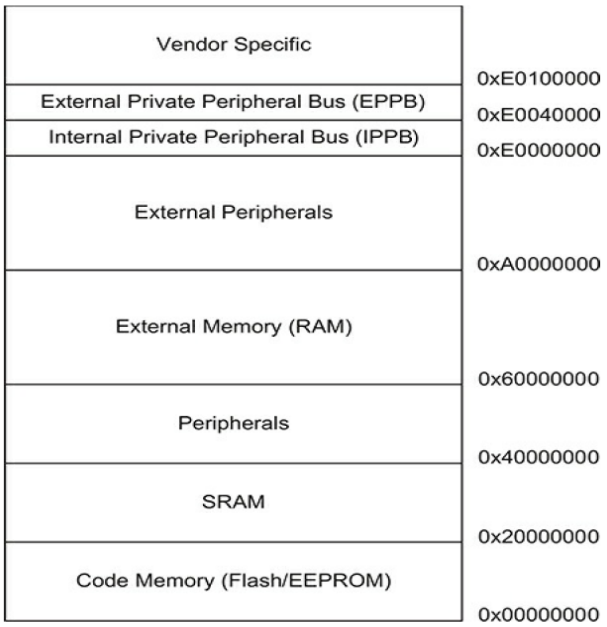


Fig 8. Memory Map for ARM Cortex Mx.

B. Exceptions and Interrupts

Exception states:

Each exception is in one of the following states:

Inactive: The exception is not active and not pending.

Pending: The exception is waiting to be serviced by the processor.

Active: The exception is being serviced by the processor but has not completed.

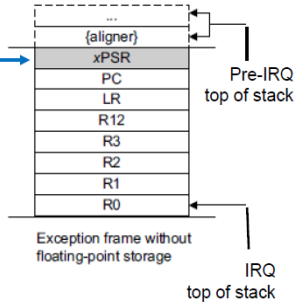
Active and pending -The exception is being serviced by the processor and there is a pending exception from the same source.

An interrupt request from a peripheral or from software can change the state of the corresponding interrupt to pending.

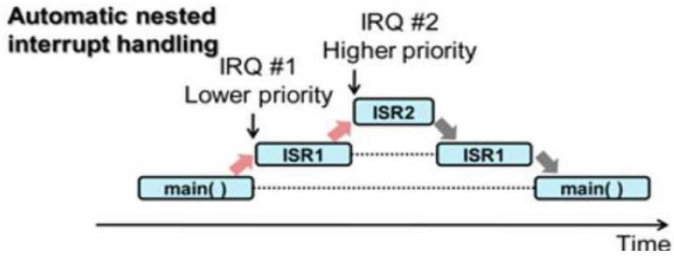
An exception handler can interrupt (pre-empt) the execution of another exception handler. In this case both exceptions are in the active state.

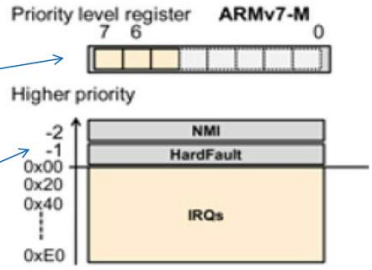
Interrupt Processing in ARM Cortex Processor

1. Interrupt signal detected by CPU
2. Suspend main program execution
   - finish current instruction
   - save CPU state (push registers onto stack)
   - set LR to 0xFFFFFFF9 (indicates interrupt return)
   - set IPSR to interrupt number
   - load PC with ISR address from vector table
3. Execute interrupt service routine (ISR)
   - save other registers to be used [1]
   - clear the "flag" that requested the interrupt
   - perform the requested service
   - communicate with other routines via global variables
   - restore any registers saved by the ISR [1]
4. Return to and resume main program by executing ***BX LR***
   - saved state is restored from the stack, including PC



Priority in Interrupts





Jp to 256 priority levels
- 8-bit priority value
- Implementations may use fewer bits
  STM32L4xx uses upper 4 bits of each priority byte => 16 levels
- NMI & HardFault priorities are fixed

Role of Nested Vector Interrupt controller

Interrupt signal: from device to CPU

In each peripheral device:

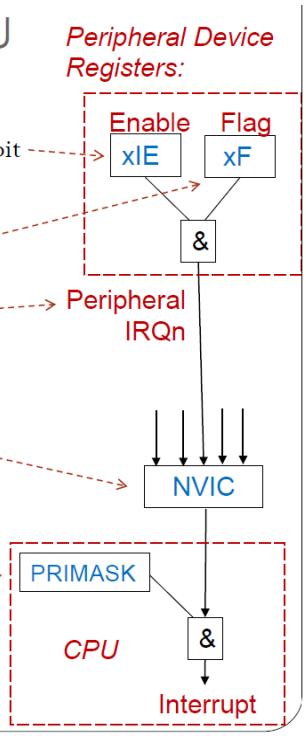
- Each potential interrupt source has a separate **arm (enable)** bit
  - Set for devices from which interrupts, are to be accepted
  - Clear to prevent the peripheral from interrupting the CPU
- Each potential interrupt source has a separate **flag** bit
  - hardware sets the flag when an "event" occurs
  - Interrupt request = (flag & enable)
  - ISR software must clear the flag to acknowledge the request
  - test flags in software if interrupts not desired

Nested Vectored Interrupt Controller (NVIC)

- Receives all interrupt requests
- Each has an enable bit and a priority within the VIC
- Highest priority enabled interrupt sent to the CPU

Within the CPU:

- Global interrupt enable bit in PRIMASK register
- Interrupt if priority of IRQ < that of current thread
- Access interrupt vector table with IRQ#



15